National Institute of Technology Tiruchirappalli



Session: July 2026

Product Development

VISUAL CAST DEFECT INSPECTION

Submitted in partial fulfilment of the requirements for the course

PRPC21 Manufacturing Tooling and Automated Inspection

Submitted by

114124024 Ganesh Bhavaraju

114124056 Kruthika

114124064 Naqiya

114124066 Nitesh

Submitted to

Dr. Mallikarjuna

Course Faculty

Department of Production Engineering
National Institute of Technology, Tiruchirappalli
India

Visual Cast Defect Inspection: An On-Device System for Detecting, Classifying and Remedying Casting Defects

Ganesh Bhavaraju[1], Kruthika[1], Naqiya[1] and Nitesh[1]
[1]Department of Production Engineering
National Institute of Technology Tiruchirappalli
Tiruchirappalli, Tamil Nadu 620015, India
{Corresponding author's email: niteshk202006official@gmail.com}

Abstract

Visual inspection of metal castings is still done by eye in most small and medium foundries, which is slow, inconsistent and leaves no record. This project presents Visual Cast Defect Inspection, a phone application that inspects a casting from a single camera photo, flags defective parts, names the likely defect type and gives the operator pre-written corrective actions. All processing runs inside the phone's browser, so no GPU, server or network connection is needed and no image leaves the device. Three inspection modes are provided: trained neural networks for a known product, a training-free comparison against a few photos of good parts, and a comparison against the part's CAD model that never mistakes a designed hole for a defect. The networks were trained on public datasets from Kaggle and Roboflow, including real cast images taken on an industrial foundry inspection line, and the training-free modes were tested on photo-realistic CAD renders. The whole-part classifier is 99.7% accurate on 715 test photos, the close-up classifier names eight defect classes at 90.4% accuracy, and the CAD mode typed all 139 detected validation defects correctly.

Keywords

Casting defects; Automated visual inspection; Deep learning; Anomaly detection; CAD-based inspection; On-device inference

1. Introduction

1.1 Background

Casting is one of the oldest and most widely used ways of making metal parts. Molten metal is poured into a mould and left to solidify into the shape of the cavity, which makes it the cheapest route to complex shapes such as pump impellers, valve bodies, brackets, gearbox housings, connecting rods and engine blocks [1, 2]. India is the second largest producer of castings in the world, and most of its several thousand foundries are small and medium enterprises. The process is, however, sensitive to small changes in sand moisture, pouring temperature, gating design and handling, and every foundry produces some defective parts.

A defective casting that reaches the customer can leak, crack in service or fail an assembly, and a whole order may be rejected because of a few bad parts. Final inspection is therefore an essential step of every foundry, and for surface and shape defects it is almost always done visually.

1.2 Casting defects

A casting defect is any unwanted irregularity in the finished part. Foundry literature groups them into families by their cause [1, 2]:

- **Gas defects:** blowholes, pinholes and porosity, caused by gas trapped from moist sand, cores, binders or a gassy melt. They appear as smooth, round cavities.

- **Shrinkage defects:** rough, jagged or spongy cavities where the metal contracted while solidifying and was not fed by a riser, usually at a thick section.
- **Cracks and hot tears:** fractures caused by contraction stresses when the mould or core resists the shrinking metal, or by early shake-out and rough handling.
- **Pouring defects:** misruns, where the metal froze before filling the mould, and cold shuts, where two streams of metal met but did not fuse.
- **Mould material defects:** sand inclusions, scabs and swells from loose, weak or unevenly rammed sand.
- **Finishing and handling defects:** flash and burrs along the parting line, chipped edges, scratches and holes blocked by missing or broken cores.

1.3 Problem statement

Most small and medium foundries still inspect castings by eye. Manual inspection is slow and tiring, two inspectors often grade the same part differently, accuracy drops over a long shift, and there is usually no record of why a part was rejected. Commercial machine vision systems exist, but they need dedicated cameras, lighting rigs, a large set of labelled defect images for every product and often a GPU server, which puts them out of reach of most shops.

The problem this project addresses is therefore: **to build an inspection tool that a foundry can use with only a phone, that works for new part types without collecting thousands of defect images, and that tells the operator what the defect is and what to do about it.**

1.4 Objectives

1. Detect defective castings from a phone photo, on the device itself, with no server or GPU.
2. Support new part types quickly, either from a few photos of good parts or from the part's CAD model.
3. Never flag designed features such as holes as defects, and detect when the wrong part or the wrong face is shown to the camera.
4. Name the likely defect type and show where it is on the part.
5. Give pre-written causes, corrective actions and a disposition for every defect type.
6. Produce an inspection record: a PDF report for each part and a CSV log for the shift.

2. Literature Survey

Automated casting inspection has moved from hand-tuned image processing to deep learning over the last two decades. The relevant work falls into five groups, summarised in Table 1.

2.1 Radiography and classical machine vision

Early automated systems inspected castings with X-ray radioscopy, because internal porosity and shrinkage are invisible from the outside. Mery and Filbert [3] detected flaws in aluminium wheels by segmenting potential defects in each X-ray image and tracking them over an image sequence, which removed most false alarms caused by the part's own structure. The GDXray database [4] later made thousands of such X-ray images public. These methods rely on hand-designed features and thresholds that have to be re-tuned for every new part, and X-ray equipment is far beyond the budget of a small foundry.

2.2 Supervised deep learning

Convolutional neural networks learn their features from labelled examples. Ferguson et al. [5] showed that networks pre-trained on ImageNet [6] and fine-tuned on a few thousand X-ray images detect and segment casting defects far better than hand-designed features, which is the transfer-learning approach used in this project. For visible surface defects, the Kaggle dataset of

pump impellers photographed on the inspection line of an Indian foundry [7] has become the standard benchmark. Networks such as VGG16 [8], ResNet50 [9] and Vision Transformers [10] reach 99% or more defect-or-OK accuracy on it. Object detectors based on YOLO [11], such as DES-YOLO [12] and SLGA-YOLO [13], go further and localise and name individual blowholes, cracks, porosity and sand holes. The limitation of all supervised methods is data: every new part or defect class needs hundreds of labelled images. To make such classifiers trustworthy, class activation maps (CAM) [14] and Grad-CAM [15] highlight the image regions that drove the decision.

2.3 Unsupervised anomaly detection

Anomaly detection learns only from good parts and flags anything that differs from them. The MVTec AD benchmark [16] made this setting popular for industrial inspection. PaDiM [17] models the features of each image patch with a Gaussian distribution, and PatchCore [18] stores patch features of good images from an ImageNet network in a memory bank and scores each test patch by its distance to the nearest stored patch. PatchCore reaches state-of-the-art results on MVTec AD without a single defect image. These methods, however, cannot tell which differences matter (a new background or a shadow is as anomalous as a crack), and they do not name the defect.

2.4 CAD-based and pose-aware inspection

A third line of work compares a photo with the object's 3D model rendered from the camera's viewpoint, so designed features are handled correctly whatever the pose. PAD and its MAD dataset with the OmniposeAD method [19], SplatPose [20] and Looking 3D [21] detect anomalies on objects seen from arbitrary poses. Most of them need a calibrated multi-view rig or a GPU to reconstruct or render the object, which is not available on a shop floor.

2.5 Foundry diagnosis

Foundry handbooks [1, 2] diagnose defects by appearance: smooth round cavities indicate gas, rough dendritic ones shrinkage, rounded unfilled edges a misrun and bright sharp fractures breakage. Each family has well-known causes and corrective actions, but none of the automated systems above links its output to this knowledge.

Table 1 Comparison of approaches to automated casting inspection

Approach	Examples	Needs	Strength	Limitation
Radiography with classical vision	[3, 4]	X-ray unit, tuned thresholds	Sees internal defects	Costly, re-tuned per part
Supervised classification or detection	[5, 12, 13]	Hundreds of labelled images per part	Names the defect	New parts need new data
Unsupervised anomaly detection	[17, 18]	Good parts only	No defect images needed	No defect type; sensitive to background
CAD or pose-aware comparison	[19, 20, 21]	3D model, often GPU or multi-view rig	Understands designed features	Special hardware
This project	Sections 3.4 to 3.6	A phone; a few good photos or a CAD file	All three, typed, with remedies	See Section 4.6

2.6 Research gap

Supervised models name defects but need large labelled sets; anomaly detection needs no defects but cannot name them; CAD methods understand geometry but need special hardware; and none of them tells the operator what to do next. This project combines all three approaches in one phone application, adds defect typing to the two training-free modes, and links every defect type to foundry guidance on causes and corrective actions.

3. Methodology

3.1 Overall approach

The application, called Visual Cast Defect Inspection, offers three inspection modes as three tabs, so that a foundry can choose the one that suits the part and the data it has:

- **Base model:** trained neural networks for a known product. A whole-part classifier says defective or OK, and a close-up classifier names the defect type when the operator photographs the defect from close up.
- **Reference images:** a training-free comparison. The operator photographs 1 to 20 good parts once and every later part is compared with them.
- **Reference CAD:** a training-free comparison with the part's CAD model, which knows where the designed holes are and what the outline should be in millimetres.

All three tabs end the same way: a verdict, a map of where the evidence is, the likely defect type, a remedy card and an inspection record.

3.2 Datasets

The project uses three kinds of data: real industrial casting images from Kaggle, real labelled defect photos from Roboflow, and synthetic photos rendered from CAD models to test the training-free modes. The sources are listed in Table 2.

Table 2 Datasets used in the project

Dataset	Content	Images	Used for
Casting product image data [7] (Kaggle)	Pump impellers photographed on the inspection line of Pilot Technocast, Rajkot; defective or OK	7,348 (from 1,300 originals)	Whole-part model; good-surface crops
casing_defect_detection [22] (Roboflow)	Boxes on blowholes, shrinkage, scratches, mould defects and burrs	1,004	Close-up model
casting_dataset_kaggle [22] (Roboflow)	Blowholes, shrinkage, cracks, misruns, cold shots, flash, scars, scratches	781	Close-up model
casting-defects and casting-dataset-2 [22] (Roboflow)	Blow holes, cracks, burrs, scratches, shrinkage, porosity	281	Close-up model
SIFA [23] (Kaggle)	Real sand castings photographed in a foundry	1,080	Good-surface crops
CAD renders (this project)	Five parts, good and defective, on coloured mats	649 plus test kits	Testing tabs 2 and 3

3.2.1 Industrial images

The core of the training data is real cast images taken in industry. The Kaggle casting dataset [7] was captured on the production inspection line of Pilot Technocast, a foundry in Rajkot, Gujarat, and shows the top face of submersible pump impellers under stable lighting, in grayscale, labelled defective or OK. The Roboflow projects [22] are hand annotations of the same industrial impeller photos in which every defect is marked with a bounding box and a class name (Figure 1). The close-up defect crops are therefore cut from real castings photographed on a real inspection line. The SIFA set [23] adds photos of sand castings taken in a working foundry, used as examples of good cast surfaces.

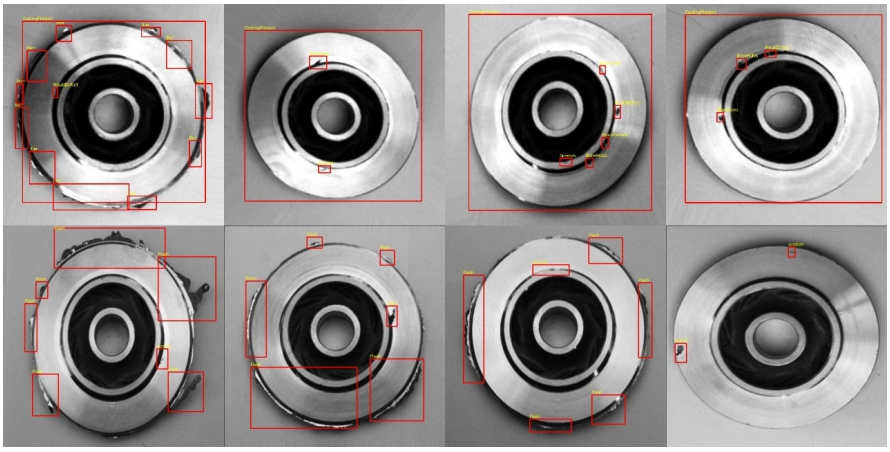


Figure 1 Industrial impeller photos from the Roboflow projects with their defect annotations (burr, flash, blowholes, mould defect, scratch)

3.2.2 Preparing the close-up defect set

Each labelled box was turned into a square crop centred on the defect, 1.6 times the size of the box, which is what a phone close-up of that defect looks like. The preparation steps were:

1. **De-duplication.** Roboflow exports contain augmented copies of each photo (three per photo in one set). Copies were removed by matching the original file name, and photos shared between projects were used once.
2. **Class merging.** Equivalent labels from different projects were merged into eight classes: BlowHoles, Blowholes, blow hole and Porosity became *gas*; Misrun and Cold shot became *misrun*; Burr, Flash and Plash became *flash/burr*; MouldDefect and Scar became *mould/surface*.
3. **Good surface.** Crops of the same sizes were cut at random positions from defect-free impellers (four per photo) and from the SIFA sand castings.
4. **Split by photo.** Train, validation and test sets (70/15/15) were assigned by a hash of the source photo name, so crops from one photo never appear on both sides of the split.
5. **Rejected data.** One Kaggle set with 12 defect classes turned out to be synthetic (black shapes painted onto a single impeller photo) and was not used. One Roboflow project whose boxes did not line up with the defects was also excluded.

The resulting class counts are given in Table 3, and sample crops are shown in Figure 2. The set is strongly imbalanced: flash and burr crops outnumber cracks and misruns about twenty to one, which the training procedure has to compensate for (Section 3.4.2).

Table 3 Close-up crops per class and split

Class	Train	Validation	Test
Flash / burr	2,318	556	599
Good surface	1,718	375	289
Gas porosity / blowhole	810	208	177
Shrinkage	741	149	159
Scratch	728	156	184
Mould / sand defect	427	90	80
Crack	123	32	26
Misrun / cold shut	119	26	20
Total	**6,984**	**1,592**	**1,534**

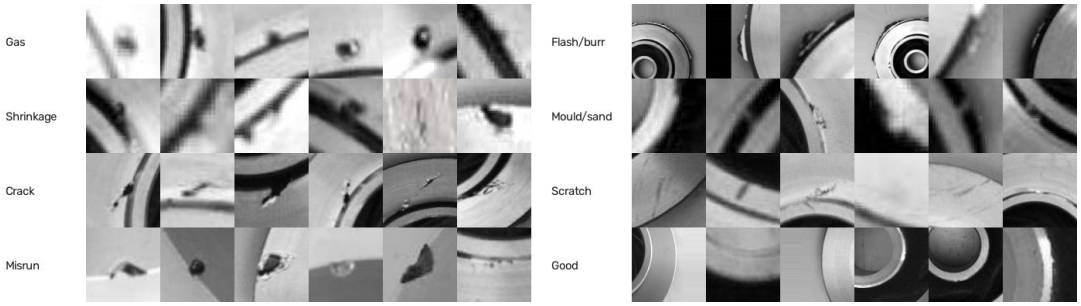


Figure 2 Close-up training crops, six per class, cut from the labelled defect boxes and from defect-free parts

3.2.3 Synthetic test photos

Real photos of the same part with known, controlled defects are hard to collect, so the two training-free modes were tested on photo-realistic renders. Parametric CAD models of five parts were built in code: an impeller with 6 blades and 3 balance holes, an L-bracket, a pump housing, a connecting rod and a 6004-2RS ball bearing (20 mm bore, 42 mm outside diameter, 12 mm wide). Defective variants were made by Boolean operations on the meshes: through blowholes, blind pits, blocked holes, chipped rims, broken blades, cracks, flash, misruns and shrinkage cavities. Each model was rendered in a headless browser with a physically based metal material, a procedural sand-cast surface texture, a coloured mat, random rotation, position, tilt and lighting, and was then given sensor noise, blur and JPEG compression. The renderer is deliberately different from the one the application uses internally, so that the tests are not circular.

3.3 System architecture

The application is a Progressive Web App (PWA): a single web page that installs on a phone like an app, works offline after the first visit and runs every model on the device (Figure 3). The server only delivers static files; no photo is ever uploaded.

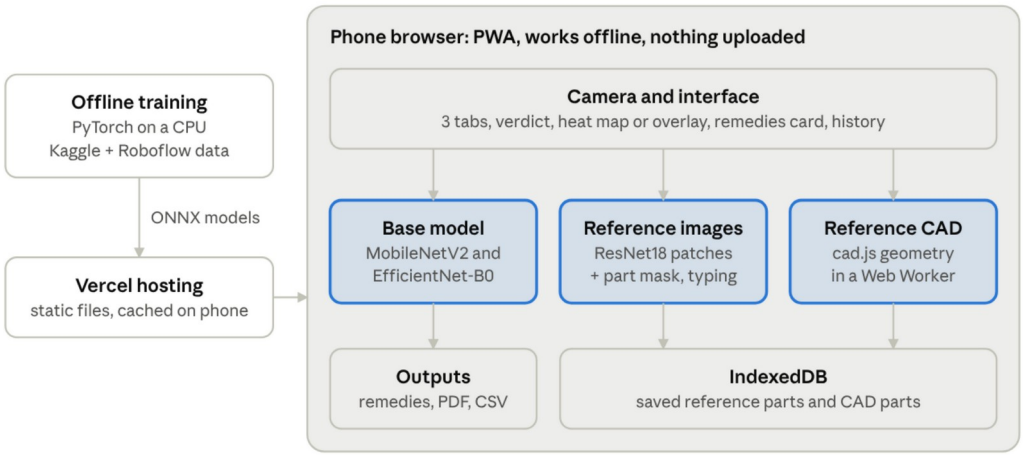


Figure 3 System architecture. The models are trained offline and delivered once as static files; inspection, storage and reporting all happen in the phone's browser

3.3.1 Technology stack

The technologies used in each layer are listed in Table 4.

Table 4 Technology stack

Layer	Technology
User interface	HTML, CSS and plain JavaScript, no framework; a Web Worker for heavy computation
Neural networks in the browser	ONNX Runtime Web 1.20 running on WebAssembly [24]
Geometry engine	cad.js, written for this project with no dependencies (about 860 lines)
Storage and offline use	IndexedDB for saved parts, localStorage, a service worker for offline caching
Reports	html2canvas and jsPDF for the PDF report; CSV export of the log
Training	Python, PyTorch and timm [25] on a CPU; models exported to ONNX
Testing	Node.js, Playwright with headless Chromium, three.js and manifold3d for the CAD renders
Hosting	Vercel static hosting from GitHub, with a preview link for every branch

3.3.2 Why on the device

Running in the browser means the application needs no GPU, no server and no network connection on the shop floor, and photos of customer parts never leave the phone. The models were chosen to be small: the whole-part classifier is 8.9 MB, the close-up classifier 16 MB and the reference-image backbone 11 MB. They are downloaded once and cached by the service worker.

3.3.3 Inspection flow

1. The camera frame, or a photo chosen from the gallery, is centre-cropped to a square and resized to 448 × 448 pixels. Corner brackets on the live view show exactly what the model will see.
2. The active tab's engine inspects the image: a neural classifier (Base model), a patch comparison against good parts (Reference images) or a comparison with the CAD (Reference CAD).

3. The result is a verdict (PASS, REJECT, NO MATCH or WRONG PART), a heat map or overlay showing where the evidence is, and for rejects a list of typed findings.
4. Each defect type is mapped to a pre-written remedy guide, which is shown on screen and added to the PDF report.
5. Every inspection is logged with its time, verdict, scores and defect types, and the log can be exported as CSV.

3.4 Tab 1: Base model

The Base model tab uses trained networks and asks what the operator is photographing: the **whole impeller**, handled by the defect-or-OK classifier, or a **close-up of a defect**, handled by the defect-type classifier.

3.4.1 Whole-part classifier with Grad-CAM

A MobileNetV2 network [26] pre-trained on ImageNet [6] was fine-tuned on the Kaggle impeller photos [7] to output two classes, defective and OK. Training used 224 × 224 inputs, random flips, rotations of up to 20 degrees and brightness and contrast jitter, with the Adam optimiser [27] at a learning rate of 3×10^{-4} for 4 epochs. A part is rejected when the defect probability exceeds 0.5.

To show why a part was rejected, the application draws a Grad-CAM heat map [15]. Because the network ends in global average pooling followed by a single linear layer, the gradient of the defect score with respect to each feature map equals that class's weight, and the heat map reduces to the class activation map [14] of Equation (1). It can therefore be computed on the phone from the last feature maps without any back-propagation:

$$M(x, y) = \mathrm{ReLU}\left(\sum_k w_k^{\mathrm{defect}} A_k(x, y) \right) \tag{1}$$

where $A_k(x, y)$ is the k-th feature map of the last convolution layer and w_k is the weight of the linear layer for the defect class. The training script checks that this matches Grad-CAM computed with automatic differentiation (cosine similarity above 0.99) and that the exported ONNX model gives the same outputs as PyTorch.

3.4.2 Close-up defect-type classifier

The whole-part model only says defective or OK. To name the defect, a second network classifies a close-up photo, in which the defect fills the frame, into eight classes: gas porosity or blowhole, shrinkage, crack, misrun or cold shut, flash or burr, mould or sand defect, scratch, and good surface.

Model. EfficientNet-B0 [28] with ImageNet weights from the timm library [25] and an input of 128 × 128 pixels was chosen as the best trade-off between accuracy and size for a phone. EfficientNet-B0 also ends in pooling and a linear layer, so the same weight-based heat map shows where the predicted type was seen.

Grayscale input. All training photos are grayscale, so the application converts phone photos to grayscale before classifying them. This removes colour as a cue the model never learned and narrows the gap between the training photos and phone photos.

Class imbalance. The rare classes (crack and misrun have about 120 training crops each, flash and burr 2,318) are weighted in the cross-entropy loss by the square root of the inverse class frequency, Equation (2), where n_c is the number of training crops of class c. The square root avoids over-fitting the rare classes, which full inverse weighting would cause. Label smoothing [29] of 0.1 is added because some source annotations are inconsistent (Section 4.2).

$$\omega_c = \sqrt{\frac{\max_j n_j}{n_c}} \tag{2}$$

Training. The hyper-parameters are listed in Table 5. Because a defect has no natural up or down, crops are rotated through the full 360 degrees during training. The epoch with the best validation macro-F1 is kept. Training took about 50 minutes on a 4-core CPU with no GPU; the learning curve is shown in Figure 4.

Table 5 Training settings of the close-up classifier

Setting	Value
Backbone	EfficientNet-B0, ImageNet weights (timm efficientnet_b0.ra_in1k)
Input	128 × 128 pixels, grayscale copied to three channels, ImageNet normalisation
Optimiser	AdamW [30], weight decay 0.02
Learning rate	One-cycle schedule [31], peak 6×10^{-4}, 15% warm-up
Batch size and epochs	48 crops, 12 epochs
Loss	Class-weighted cross-entropy (Equation 2), label smoothing 0.1, dropout 0.2
Augmentation	Rotation 0 to 360 degrees, random crop of 60 to 100% of the frame, horizontal and vertical flips, brightness and contrast jitter of 35%, Gaussian blur with probability 0.3
Model selection	Best validation macro-F1 (epoch 9 of 12)

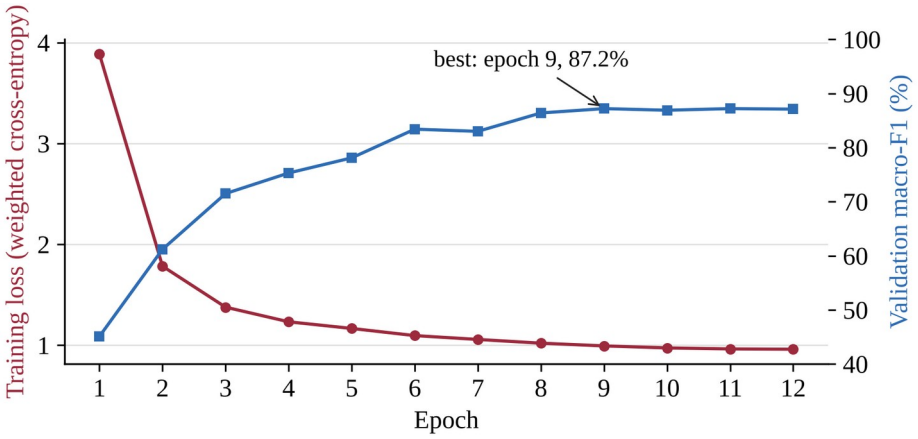


Figure 4 Training loss and validation macro-F1 of the close-up classifier per epoch

Decision rule. A part is rejected when the probability of *good surface* is below 0.5. The defect type is the most likely of the seven defect classes, and the application reports how clearly it wins: when the top type holds less than 55% of the total defect probability, the second most likely type is named as well (for example "likely gas porosity, could also be shrinkage").

Export. The model is exported to ONNX (16 MB) together with a JSON file holding the class names, input size, normalisation constants and the classifier weights used for the heat map. The export is checked against PyTorch on test images before it is deployed.

3.5 Tab 2: Reference images

The Reference images tab inspects any part type without training. The operator photographs 1 to 20 good parts once; every later photo is compared with them, and anything that looks unlike all of them is flagged. The method follows PatchCore [18], adapted to run on a phone.

3.5.1 Enrolment

1. **Patch features.** Each good photo is passed through a ResNet18 backbone [9] pre-trained on ImageNet (11 MB in ONNX form). Intermediate feature maps are pooled into a 28 × 28 grid of 128-value patch descriptors, so that each patch describes a small neighbourhood of the part.
2. **Rotations.** For round parts the operator ticks "any rotation", and each photo is also added rotated by 90, 180 and 270 degrees.
3. **Memory bank.** All patches are stored in one bank, sub-sampled to at most 30,000 patches to keep it small enough for a phone.
4. **Threshold.** Each of the first five good photos is scored against a bank built from the others (leave-one-out), and the largest of these scores becomes the threshold of Equation (4). The threshold therefore reflects how much good parts naturally vary. A Stricter/Looser slider scales it by a factor s between 0.6 and 1.4.
5. **Saving.** The profile (bank, threshold, thumbnails and six reference photos for defect typing) is stored in the phone's IndexedDB, so it survives restarts and works offline.

3.5.2 Inspection

For a new photo, each of its 784 patch descriptors p_i is matched to its nearest neighbour in the bank; a small ONNX distance model processes the bank in chunks of 6,000 patches. The map of nearest-neighbour distances is the heat map, and the anomaly score is the mean of the 10 largest distances, Equation (3):

$$\text{score} = \frac{1}{10} \sum_{i \in \text{top } 10} \min_{b \in \text{bank}} \|p_i - b\|_2 \tag{3}$$

$$\tau = s \cdot \max_{j=1\ldots5} \text{score}(I_j \mid \text{bank}_{-j}) \tag{4}$$

The verdict is REJECT when the score exceeds the threshold. A defect affects only a few patches, while a different object differs almost everywhere, so when the *median* patch distance exceeds 1.35 times that of good parts the verdict is NO MATCH ("this is not the enrolled part").

3.5.3 Ignoring the background and designed holes

A plain patch comparison treats every pixel alike. A through-hole shows whatever the part is lying on, so a new table, a sheet of paper or a shadow made good parts look defective: in a test with good bearings on different backgrounds only 2 of 12 passed, and the bearing's own bore was reported as a blocked hole. The application therefore separates the part from its background before scoring:

- **Part mask.** On a coloured mat, each pixel is scored by how much of the mat's hue it carries relative to its lightness, and the scores are split with Otsu's threshold [32]; grey metal scores near zero whether it is lit or shaded. On white, grey or black paper the mask comes from the brightness difference against the photo border, followed by a morphological opening that removes ruled lines and paper texture.
- **Repainting.** Everything outside the part, including what shows through its holes and a 2-pixel band at its edge, is repainted with one flat colour, the median background colour of the enrolled photos. This is done in enrolment and inspection alike.

Only the part's own appearance is then left to compare. New holes and chips still show, because background colour appears where good parts have metal, and a blocked hole shows metal where good parts have background.

3.5.4 Naming the defect type

A patch distance says *where* a part differs, not *how*. To name the defect, the application compares the flagged areas with the best-matching good reference photo:

1. **Registration.** The test photo and each of the six stored references are unrolled into polar images around the part's centre (360 angles × 40 rings, holding the grey level and the part mask, with the mask weighted higher so that holes decide between symmetric rotations). Circular cross-correlation gives the rotation, the median outline radius gives the scale, and a small shift search refines the position.
2. **Differences.** The best reference is warped onto the test photo, and inside each hot area of the heat map the application measures metal lost, metal gained, holes opened or filled, new dark marks and dark structure that has disappeared.
3. **Rules.** Metal lost inside the part is a through blowhole (round) or a through crack (long and thin). Metal lost at the outline is a chipped edge, or a misrun when large. Metal gained in a reference hole is a blocked hole; metal gained beyond the outline is flash (a long strip) or extra metal (compact). A new thin dark line is a crack, new dark pits are a surface blowhole, or gas porosity when there are three or more, and missing dark structure is a missing or deformed feature such as a broken blade.
4. **Filled holes.** Any designed hole that is filled is reported even when the heat map peaked elsewhere.

Findings are labelled *likely* and numbered on the heat map. Typing needs a background that the part stands out from; otherwise the application says so and gives no type rather than guessing.

3.6 Tab 3: Reference CAD

The Reference CAD tab needs no photos of good parts. The operator loads the part's CAD file once, and every photo is checked against what the CAD says the part should look like from the camera. Because the CAD contains the designed holes, they are never mistaken for defects, and a different casting or the wrong face shown to the camera is reported as WRONG PART. The engine (cad.js) was written from scratch in JavaScript with no libraries and runs in a Web Worker so that the interface stays responsive. The pipeline is shown in Figure 5.

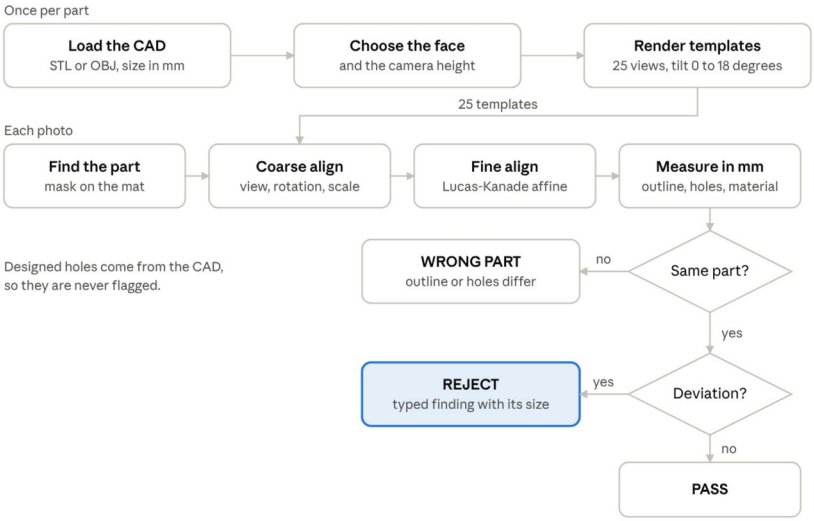


Figure 5 The CAD inspection pipeline. A wrong part is caught before any defect is reported, so a different casting is never mislabelled as defective

3.6.1 Setting up a part

1. **Load the CAD.** STL (binary or text) or OBJ. The application reports the triangle count and the size in millimetres.
2. **Pick the face** that will face the camera from six small previews (top, bottom, front, back, left, right).
3. **Enter the camera height** above the table in millimetres, so that the templates use the right perspective; tall parts look different from close up.
4. **Render templates.** The engine renders the chosen face with its own z-buffer rasteriser from 25 viewpoints (straight down, and tilted by 6, 12 and 18 degrees in 8 directions) at 320 × 320 pixels. Each template stores the silhouette, the designed through-holes and their mouths, the CAD edges with their directions and the outline. This takes about 1 to 1.5 seconds and is cached.

3.6.2 Inspecting a photo

1. **Segmentation.** The part is separated from a plain coloured mat with the hue rule of Section 3.5.3, Otsu's threshold [32], a morphological opening and closing, and the largest connected component. Tiny specks of background inside the part are filled, while real holes are kept.
2. **Coarse alignment.** The photo mask and every template are reduced to 96 × 96 pixels and the rotation (in 2-degree steps), scale and position are searched. Among near-equal candidates, the one whose hole pattern matches best is kept, which resolves symmetric parts.
3. **Fine alignment.** A Lucas-Kanade [33] affine refinement on blurred masks aligns the photo with the template to sub-pixel accuracy.
4. **Measurements** in the template frame, in millimetres: the outline match of Equation (5); a feature ratio comparing the edge strength along the CAD's interior edges with the edge strength elsewhere on the part; for each designed hole, how much background shows through its mouth; openings in the photo where the CAD has none; and material missing or extra beyond a 1 mm tolerance band around the outline.

$$\mathrm{IoU} = \frac{|S_{\mathrm{photo}} \cap S_{\mathrm{CAD}}|}{|S_{\mathrm{photo}} \cup S_{\mathrm{CAD}}|} \tag{5}$$

where S_{photo} and S_{CAD} are the photo and CAD silhouettes after alignment.

3.6.3 Decision rules

The measurements are turned into a verdict by the rules of Table 6. The wrong-part checks come first, so a different casting is never reported as a defective one.

Table 6 Decision rules of the Reference CAD tab

Check	Rule	Result if it fails
Outline	Outline match (IoU) below 90%	WRONG PART
Features	Feature ratio below 1.45	WRONG PART
Hole pattern	At least half of the checkable holes blocked, or extra openings larger than half the designed hole area	WRONG PART
Designed holes	A checkable hole less than 50% open	REJECT: blocked hole
Extra openings	An opening of 2 mm² or more where the CAD has none	REJECT: hole not in the CAD
Outline material	Missing or extra area beyond the 1 mm band of at least 6 mm² (0.2% of the part area for small parts, never under 2 mm²)	REJECT: missing or extra material

A deep or off-centre hole may show almost no background from the camera position. A hole is checked only when at least 45% of its mouth should be see-through from that view; otherwise the result states how many holes could not be checked instead of guessing.

3.6.4 Naming the defect type

Each finding is typed from where it lies relative to the CAD and from its shape in millimetres (Table 7). The shape is measured from the second moments of the region as an equivalent ellipse (length, width and elongation) and from how much of that ellipse the region fills. See-through fragments within 2.5 mm of each other are grouped, because a crack or a ragged void shows the background only in pieces.

Table 7 Defect typing rules of the Reference CAD tab

Finding	Shape	Type reported
Opening not in the CAD	Elongation of 3 or more and width under 3 mm	Through crack, with its length
Opening not in the CAD	Several fragments, or ragged	Shrinkage cavity (through)
Opening not in the CAD	Round	Blowhole (pinhole under 2 mm), with its diameter
Designed hole	Under 15% open, or metal in its mouth	Hole not formed (core missing or broken)
Designed hole	Partly open	Partly blocked hole (sand or flash in it)
Material missing at the outline	150 mm² or more, or 25 mm or longer	Broken or unfilled section (misrun)
Material missing at the outline	Smaller	Chipped edge, with its size
Material beyond the outline	Elongation of 4 or more	Flash or fin, with its length and height
Material beyond the outline	Compact	Extra metal (swell or lump)

The largest finding is listed first. The overlay draws the CAD edges in red, the designed holes in blue and the findings filled in, aligned on the photo.

3.7 Defect remedies and inspection records

Finding a defect is only useful if the operator knows what to do next. Every defect type the three tabs can report is mapped to one of 13 pre-written guides, written from standard foundry practice [1, 2]. Each guide has four parts:

- **Confirm:** what the defect looks like, so the operator can check the call by eye (for example, a blowhole has smooth, round walls while shrinkage is rough and dendritic).
- **Likely causes** in the process.
- **Fix the process:** concrete corrective actions.
- **This part:** the disposition (re-inspect with dye penetrant or X-ray, repair, re-work or scrap).

The thirteen guides and their main corrective actions are summarised in Table 8.

Table 8 Remedy guides and their main corrective actions

Guide	Main corrective actions
Gas porosity / blowhole	Drier sand and cores, more vents, degas the melt, pour through a filter
Shrinkage cavity	Resize or move risers, exothermic sleeves, chills, pour within the temperature range
Crack / hot tear	Collapsible cores, fillets at corners, longer and controlled cooling
Misrun / cold shut	Hotter, faster and uninterrupted pour; larger gates; vents at the last place to fill
Chipped or broken edge	Gentler shake-out, saw off gates and risers, padded racks
Blocked or missing hole	Core-setting check before closing, stronger cores, better decoring
Flash / fin / burr	Clamp or weight moulds, repair patterns and core prints, raise die clamp force
Swell / extra metal	Even ramming, mould coating at erosion spots, lower pouring height
Sand or slag inclusion	Blow out moulds, ceramic filters, skim slag
Scratch or handling mark	Trays and separators, check fixtures, tools and blasting media
Missing or deformed feature	Follow the breakage or misrun guide; check pattern and core box
Surface anomaly	Clean and re-inspect; mould coating for sand burn-on
Untyped defect	Identify the type by eye or with another tab, then follow that guide

When a part is rejected, a "What to do" card appears under the verdict with the guide for the defect type found; when several types were found, chips switch between them. Each inspection can be exported as a PDF quality report: the first page carries the part ID, date and time, verdict stamp, confidence gauge, the photo beside the heat map or CAD overlay, a defect-type table, the recommended action and sign-off lines; the second page is the action plan for every defect type found. The CSV log lists every inspection of the session with its time, verdict, scores and defect types, ready for a spreadsheet or a quality system.

3.8 User interface and implementation

The interface is designed for one-handed use on the shop floor (Figure 6):

- **Navigation:** a floating, rounded bar switches between the three tabs with a sliding highlight. Only GPU-friendly transform animations are used, so tab switches stay close to 50 frames per second even on a slow phone.
- **Camera and result:** the live camera view sits beside the heat map or CAD overlay. On a phone the result shrinks to a picture-in-picture corner that enlarges when tapped.
- **Verdict and remedies:** PASS in green, REJECT in red, NO MATCH or WRONG PART in amber, one line of detail, and the remedy card for rejects.
- **History:** the last ten inspections, each with a PDF button.

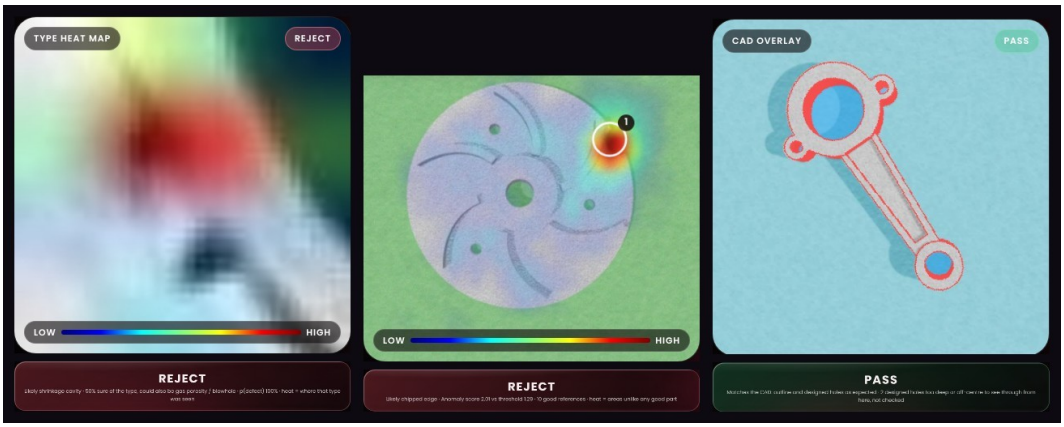


Figure 6 The three tabs in use. Left: Base model in close-up mode with the heat map and the likely type. Middle: Reference images with the difference map and the numbered finding. Right: Reference CAD with CAD edges (red) and designed holes (blue) aligned on a connecting rod

The page is about 1,150 lines of code (index.html), the geometry engine about 860 (cad.js) and the remedy guides about 100 (remedies.js). CAD rendering, CAD inspection and defect typing run in a Web Worker built from the same cad.js, which also runs under Node.js for automated tests. Saved parts live in IndexedDB, and a service worker serves the page network-first and caches the models for offline use. Every Git branch gets its own Vercel preview link, and the main branch is the production application.

4. Results and discussion

4.1 Evaluation protocol

Each tab was tested on photos it was never tuned on. The neural networks were evaluated on held-out test photos whose crops never appeared in training (Section 3.2.2). The training-free tabs were evaluated on the synthetic test kits of Section 3.2.3, including blind kits that were generated only after the rules had been fixed. Every change was also checked end to end in the real application with automated browser tests. Accuracy, recall (the share of a class that was caught), precision (the share of the calls of a class that were right) and macro-F1 (the unweighted mean of the per-class F1 scores) are reported.

4.2 Base model

The whole-part classifier is **99.7% accurate** on the 715 test photos of the Kaggle dataset: 451 of 453 defective parts were caught and none of the 262 good parts was rejected. This matches much larger networks on the same dataset (Section 2.2) with a model of only 8.9 MB.

The close-up defect-type classifier was tested on 1,534 crops from photos it never saw in training. It is **90.4% accurate** with a macro-F1 of 0.83. Used only as a defect detector, it misses

8 of 1,245 defects (0.6%) and flags 4 of 289 good surfaces (1.4%). The per-class results are shown in Figure 7 and the full confusion matrix in Figure 8.

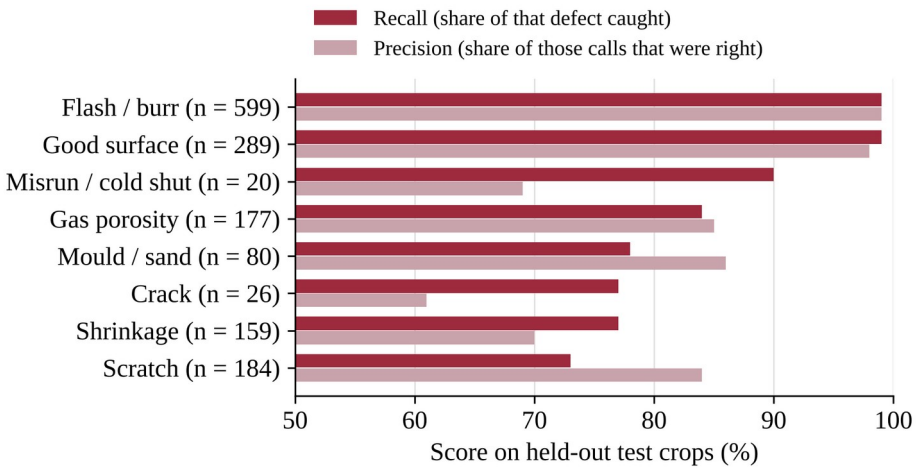


Figure 7 Recall and precision per class of the close-up classifier on the held-out test crops (the axis starts at 50%)

Flash and burr, and good surfaces, are almost perfect because they are frequent and distinct. Misrun has high recall but lower precision: with only 20 test crops, a few cracks called misrun pull its precision down. The main confusion is between the two cavity types: 20 gas crops were called shrinkage and 21 shrinkage crops were called gas. The source annotators are not consistent between these two classes either. Scratches are also mixed with shrinkage and mould marks, since a short scratch and a narrow shrinkage crack look alike at 128 pixels.

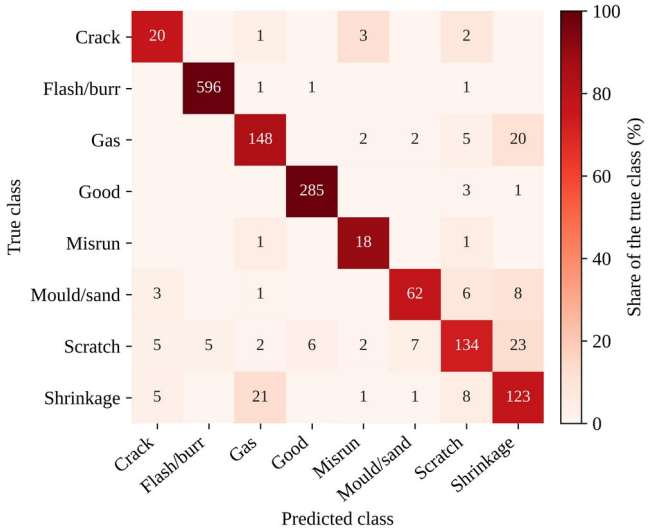


Figure 8 Confusion matrix of the close-up classifier on 1,534 held-out test crops (counts; colour shows the share of each true class)

In the real application each close-up prediction takes about 0.15 s, and 22 of 24 held-out crops sampled through the user interface were typed correctly, confirming that the browser pipeline (grayscale conversion, resizing and ONNX inference) reproduces the PyTorch results.

4.3 Reference CAD

The CAD engine was validated on 649 rendered photos of four parts, each with good parts, defective variants and wrong parts. At a fixed inspection station, 99% of good impellers (6 blades, 3 balance holes), 100% of L-brackets, 97% of pump housings and 100% of connecting rods passed.

Every detected validation defect was given the right type: **139 of 139** (46 through blowholes, 47 blocked or uncored holes and 46 chipped edges). On additional defect shapes, flash was typed correctly 8 of 8 times and misruns 8 of 8 times. Through cracks were detected 4 of 8 times, 2 of them typed as crack, and shrinkage voids were detected 5 of 8 times but typed as blowholes, because only their round core shows the mat through. Table 9 gives the results on the two blind kits, including the 6004-2RS bearing.

Table 9 Reference CAD results on the blind test kits

Test kit	Result
Connecting rod blind kit (12 photos, never used for tuning)	9 of 12 correct in the application
6004-2RS bearing: good parts	10 of 10 pass
Bearing: lump on the rim / bore blocked	4 of 4 / 4 of 4 flagged
Bearing: piece broken out / crack through the ring	3 of 4 / 3 of 4 flagged
Bearing: washer / larger 6005 bearing shown instead	2 of 4 / 1 of 4 called WRONG PART

The weak result on the 6005 bearing is expected: a 6005 has almost the same proportions as a 6004, and a phone photo without a scale reference cannot tell the two apart (Section 4.6). A photo takes about 0.8 s to inspect and a new CAD part about 1 to 1.5 s to prepare.

4.4 Reference images

The tab was tested on rendered station photos with fixed lighting: 10 good parts were enrolled, then good and defective parts were inspected (Table 10).

Table 10 Reference images results

Test set	Good parts passing	Defects flagged
Impeller station (64 defects, 10 types)	2 of 6	55 of 64
Blind impeller station	4 of 4	23 of 32
Blind bracket station	4 of 4	15 of 18
Bearing on 6 other backgrounds, before background repainting	2 of 12	16 of 16
Bearing on 6 other backgrounds, with background repainting	7 of 12	16 of 16

The background repainting of Section 3.5.3 raised the share of good bearings passing on unfamiliar backgrounds from 2 of 12 to 7 of 12, while all 16 defective bearings were still flagged and detection on the impeller and bracket sets stayed the same or improved slightly (54 to 55 of 64 and 22 to 23 of 32). The heat-map peak lay on the true defect in 53 of 60 photos, measured against twin renders without the defect. Defect typing named the right type in 50 of 64 photos on the development set and 42 of 50 on the blind sets. Blocked holes, broken blades, misruns and shrinkage were typed reliably; hairline cracks and blind pits were the hardest.

The low pass rate of good impellers at the station (2 of 6) shows the weakness of a pure appearance comparison: with only 10 enrolled parts, normal variation in lighting and texture can exceed the threshold.

4.5 Application tests and performance

Browser tests in headless Chromium drove the real application at desktop and phone screen sizes: camera permission and the no-camera path, all three tabs end to end (enrolment, CAD import, inspection by file upload), history, the PDF report and the CSV export. Tab animation stayed smooth under 6× CPU throttling (from 27 to about 49 frames per second after optimisation). On a laptop CPU in the browser a close-up classification takes about 0.15 s, a CAD inspection about 0.8 s and every engine under about 1 s per photo, with models of 8.9 MB (whole part), 16 MB (close-up) and 11 MB (reference images).

4.6 Discussion and limitations

The three modes complement each other. The Base model is the most accurate and the only one that recognises surface defects such as scratches and porosity from close up, but it only knows the product it was trained on. Reference images adapts to a new part in minutes and sees surface defects, but it cannot tell an unimportant difference from a real defect as reliably. Reference CAD needs no photos at all, never flags a designed hole and measures in millimetres, but it only sees the outline and the through-holes. The main limitations are:

- **One product family for training.** Every labelled defect photo shows the same grayscale impeller, with defects about 50 pixels across. Close-ups of other castings will be less accurate until the model is retrained with some of the user's own photos.
- **Hard classes.** Gas porosity and shrinkage are often confused, and crack and misrun had only about 120 training examples each. Corrosion and seal damage are outside the eight classes.
- **Outline and through-holes only in the CAD mode.** Surface cracks, pits, chips that do not go through the part's full height and changes to raised interior features are not judged there.
- **No absolute scale.** A phone photo has no scale reference, so a geometrically similar part (a 6005 bearing in place of a 6004) can pass the CAD check.
- **Background and validation.** The CAD tab needs a coloured mat, and the training-free tabs were validated mostly on rendered photos.

5. Conclusion

Visual Cast Defect Inspection shows that useful casting inspection can run on an ordinary phone, with no GPU, no server and no images leaving the device. The three tabs cover the three situations a foundry meets:

- **Base model:** trained networks for a known product. The whole-part model is 99.7% accurate on its test set, and the close-up model, trained on real industrial cast images from Kaggle and Roboflow, names eight defect classes at 90.4% accuracy while missing only 0.6% of defects.
- **Reference images:** a new part learned from a few good photos, made robust to the background and to what shows through designed holes.
- **Reference CAD:** a part checked against its CAD model. Designed holes are never flagged, wrong parts are caught, deviations are measured in millimetres, and all 139 detected validation defects were typed correctly.

All three tabs name the likely defect type, show where it is and link it to pre-written causes, corrective actions and a disposition, and every inspection ends as a PDF report and a CSV log. The project thus meets the six objectives set in Section 1.4.

The main next step is more real-world data. The planned future work is:

1. Retrain the close-up classifier with 20 to 30 labelled close-ups per class from a partner foundry, and add corrosion and seal-damage classes.
2. Add a printed scale marker on the mat so that the CAD mode can measure absolute size.
3. Run the Reference images surface check inside the CAD outline for parts that have both.
4. Design a fixed station with a ring light and a stand, and add STEP import and two-sided inspection.
5. Share the inspection history across phones for a supervisor dashboard.

References

[1] J. Campbell, Complete Casting Handbook: Metal Casting Processes, Metallurgy, Techniques and Design, 2nd ed. Oxford: Butterworth-Heinemann, 2015.

[2] P. N. Rao, Manufacturing Technology, Vol. 1: Foundry, Forming and Welding, 5th ed. New Delhi: McGraw Hill Education, 2018.

[3] D. Mery and D. Filbert, "Automated flaw detection in aluminum castings based on the tracking of potential defects in a radioscopic image sequence," IEEE Transactions on Robotics and Automation, vol. 18, no. 6, pp. 890-901, 2002.

[4] D. Mery, V. Riffo, U. Zscherpel, G. Mondragon, I. Lillo, I. Zuccar, H. Lobel and M. Carrasco, "GDXray: The database of X-ray images for nondestructive testing," Journal of Nondestructive Evaluation, vol. 34, no. 4, p. 42, 2015.

[5] M. Ferguson, R. Ak, Y.-T. T. Lee and K. H. Law, "Detection and segmentation of manufacturing defects with convolutional neural networks and transfer learning," Smart and Sustainable Manufacturing Systems, vol. 2, no. 1, pp. 137-164, 2018.

[6] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248-255.

[7] R. Dabhi, "Casting product image data for quality inspection," Kaggle, 2020. [Online]. Available: https://www.kaggle.com/datasets/ravirajsinh45/real-life-industrial-dataset-of-casting-product

[8] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in Proc. International Conference on Learning Representations (ICLR), 2015.

[9] K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 770-778.

[10] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in Proc. International Conference on Learning Representations (ICLR), 2021.

[11] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.

[12] "DES-YOLO: A novel model for real-time detection of casting surface defects," PeerJ Computer Science, 2024. [Online]. Available: https://peerj.com/articles/cs-2224/

[13] "SLGA-YOLO: A lightweight castings surface defect detection method based on fusion-enhanced attention mechanism and self-architecture," Sensors, 2024. [Online]. Available: https://www.ncbi.nlm.nih.gov/pmc/articles/PMC11244307/

[14] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva and A. Torralba, "Learning deep features for discriminative localization," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2921-2929.

[15] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE International Conference on Computer Vision (ICCV), 2017, pp. 618-626.

[16] P. Bergmann, M. Fauser, D. Sattlegger and C. Steger, "MVTec AD: A comprehensive real-world dataset for unsupervised anomaly detection," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2019, pp. 9592-9600.

[17] T. Defard, A. Setkov, A. Loesch and R. Audigier, "PaDiM: A patch distribution modeling framework for anomaly detection and localization," in Proc. ICPR International Workshops and Challenges, 2021, pp. 475-489.

[18] K. Roth, L. Pemula, J. Zepeda, B. Scholkopf, T. Brox and P. Gehler, "Towards total recall in industrial anomaly detection," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 14318-14328.

[19] Q. Zhou et al., "PAD: A dataset and benchmark for pose-agnostic anomaly detection," in Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, 2023. arXiv:2310.07716.

[20] M. Kruse et al., "SplatPose & Detect: Pose-agnostic 3D anomaly detection," in Proc. IEEE/CVF CVPR Workshops, 2024. arXiv:2404.06832.

[21] "Looking 3D: Anomaly detection with 2D-3D alignment," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024. arXiv:2406.19393.

[22] Roboflow Universe casting defect datasets (CC BY 4.0): casing_defect_detection, casting_dataset_kaggle, casting-dataset-2 and casting-defects-onf6n-xt28x. [Online]. Available: https://universe.roboflow.com

[23] "Sandcast Images for Foundry Automation (SIFA)," Kaggle. [Online]. Available: https://www.kaggle.com/datasets/martellbell17/sandcast-images-for-foundry-automation-sifa

[24] Microsoft, "ONNX Runtime Web," 2024. [Online]. Available: https://onnxruntime.ai

[25] R. Wightman, "PyTorch Image Models (timm)," GitHub repository, 2019. [Online]. Available: https://github.com/huggingface/pytorch-image-models

[26] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 4510-4520.

[27] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. International Conference on Learning Representations (ICLR), 2015.

[28] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. International Conference on Machine Learning (ICML), 2019, pp. 6105-6114.

[29] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, "Rethinking the Inception architecture for computer vision," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 2818-2826.

[30] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. International Conference on Learning Representations (ICLR), 2019.

[31] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," in Proc. SPIE Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications, vol. 11006, 2019.

[32] N. Otsu, "A threshold selection method from gray-level histograms," IEEE Transactions on Systems, Man, and Cybernetics, vol. 9, no. 1, pp. 62-66, 1979.

[33] B. D. Lucas and T. Kanade, "An iterative image registration technique with an application to stereo vision," in Proc. International Joint Conference on Artificial Intelligence (IJCAI), 1981, pp. 674-679.